

LOST ECO — Guía de sustentación y documentación técnica

Proyecto: Lost Eco (videojuego 2D educativo)

Motor: Godot Engine 4.6

Tema central: acoso escolar (bullying) y resolución no violenta

Índice

1. [Qué decir en la sustentación \(guion\)](#)
 2. [Requerimientos del proyecto](#)
 3. [Descripción del juego](#)
 4. [Flujo de la campaña](#)
 5. [Mecánicas por nivel](#)
 6. [Arquitectura en Godot](#)
 7. [Cómo funciona el código](#)
 8. [Patrones de diseño](#)
 9. [Demostración sugerida](#)
 10. [Preguntas frecuentes del jurado](#)
-

1. Qué decir en la sustentación (guion)

Apertura (1–2 minutos)

«Buenos días/tardes. Presentamos **Lost Eco**, un videojuego 2D desarrollado en **Godot 4.6** cuyo objetivo es sensibilizar sobre el **acoso escolar** a través de la experiencia de juego, no de un discurso directo.

El protagonista es **Alex**, un adolescente que recorre un bosque simbólico. Cada zona representa una etapa emocional: el laberinto de las palabras, el pantano de la paciencia, la cueva del espejo y el río hacia la reconciliación. Al final llega al **Claro**, donde puede hablar con **Mateo**.

La premisa pedagógica es clara: **no se gana peleando**, sino escuchando, completando objetivos y, en el clímax, **sanando al jefe del espejo** con empatía ([E] o gesto de paz [Q]), no con violencia.»

Problema y justificación (1 minuto)

«El bullying afecta la autoestima, el rendimiento y la salud mental. Muchos materiales son folletos o charlas pasivas. Nosotros proponemos un **juego corto, accesible y memorable**, donde el

jugador vive las consecuencias del acoso (sombras que dañan, mensajes de reflexión) y practica alternativas: pulso de luz defensivo, paciencia en el pantano, lectura de memorias en la cueva.»

Requerimientos cumplidos (1–2 minutos)

«Cumplimos requerimientos **funcionales**: menú, cuatro zonas jugables, HUD con vidas y objetivos, pausa, game over, transiciones entre niveles, textos narrativos y cinemáticas.

En **técnicos**: motor Godot 4.6, resolución pixel-art 320×180 escalada a 1280×720, scripts en GDScript, arquitectura con autoloads (servicios globales) y escenas por nivel.

En **no funcionales**: mensajes claros al entrar a cada zona (`ZoneMissionBriefs`), accesibilidad parcial (daltonismo en opciones), y código organizado por carpetas (mapa, personaje, enemigos, `autoLoads`).»

Demostración en vivo (3–5 minutos)

Seguir el orden de la [sección 9](#). Mientras juegas, comenta:

- «Aquí el jugador ve la **misión** con pasos numerados — pulsa E para continuar.»
- «Las **sombras** quitan vida; el **pulso [J]** empuja sin matar.»
- «En el pantano hay que **mantener E** en los pilares — mecánica distinta al laberinto.»
- «En la cueva, cristales, sellos y ecos desbloquean la **Sala del Espejo** y el jefe se sana sin atacar.»

Arquitectura y código (2–3 minutos)

«Godot organiza el proyecto en **escenas** (.tscn) y **scripts** (.gd). Usamos **autoloads** — singletons — para vidas, diálogos y cambio de escena, así no duplicamos lógica en cada nivel.

Cada zona tiene un mapa en **texto** (matriz de caracteres): # pared, E eco, S inicio. Eso es **data-driven**: cambiar el nivel sin redibujar en el editor.

Aplicamos patrones reconocibles: **State** en el menú, **Observer** con señales (`health_changed`), **Factory/Bootstrap** para crear HUD y enemigos. No es MVC puro; es el estilo habitual en Godot: escenas con script monolítico por zona más servicios globales.»

Cierre (30 segundos)

«En resumen: Lost Eco es un juego con narrativa coherente, mecánicas variadas por nivel, código modular en Godot y un mensaje final de **empatía y diálogo**. Quedamos atentos a sus preguntas. Gracias.»

2. Requerimientos del proyecto

2.1 Requerimientos funcionales (RF)

ID	Requerimiento	Implementación
RF-01	Menú principal con Jugar, Opciones y Salir	scenes/menu.tscn, MenuStateMachine.gd
RF-02	Campaña de 4 zonas + nivel final (Claro)	Zone1-Zone4, Clearing.tscn
RF-03	Control del personaje Alex (movimiento)	Alex.gd, acciones move_* en project.godot
RF-04	Interacción con objetos ([E])	Ecos, pilares, cristales, sellos, jefe
RF-05	Sistema de vidas (3) y game over	GameManager.gd
RF-06	Puntuación y progreso por zona	score, QuestManager.gd
RF-07	HUD: vidas, objetivo, cargas de luz	PremiumZoneHUD.gd
RF-08	Minimapa con niebla de exploración	ZoneMinimap.gd
RF-09	Enemigos que dañan al jugador	ShadowEnemyVisual, EnemyBehavior
RF-10	Pulso de luz defensivo ([J])	Lógica en cada ZoneX_*.gd
RF-11	Gesto de paz / sin arma ([Q])	Alex.peace_gesture(), relevante en Zona 3
RF-12	Textos narrativos (bullying)	StoryReflections.gd, DialogueManager
RF-13	Intro de misión al entrar a cada zona	ZoneMissionBriefs.gd
RF-14	Cinemáticas entre menú y niveles	ZoneCinematicDirector.gd, SceneTransition.gd
RF-15	Pausa con ESC	GameManager.open_pause()
RF-16	Transición suave entre escenas	Fade en SceneTransition.gd
RF-17	Opciones (audio, resolución, daltonismo)	SettingsManager.gd
RF-18	Restaurar vidas al completar zona	GameManager.on_zone_completed()
RF-19	Jefe final sanable sin violencia	Zone3_Cave.gd, MirrorBossVisual.gd
RF-20	Diálogo final con Mateo	Clearing.gd

2.2 Requerimientos no funcionales (RNF)

ID	Requerimiento	Detalle
RNF-01	Rendimiento en PC modesta	Pixel art 320×180, pocos nodos por tile
RNF-02	Legibilidad de UI	HUD compacto, intros con pasos numerados
RNF-03	Mantenibilidad	Carpetas por dominio; autoloads desacoplados
RNF-04	Portabilidad del motor	Exportable desde Godot 4.6 (Windows principal)
RNF-05	Coherencia narrativa	Mismo tono en reflexiones y cinemáticas
RNF-06	Variación de gameplay	Mecánica distinta en cada zona (no solo “recoger X”)
RNF-07	Persistencia de ajustes	<code>user://settings.cfg</code>

2.3 Requerimientos técnicos (RT)

ID	Requerimiento	Valor
RT-01	Motor	Godot 4.6
RT-02	Lenguaje	GDScript
RT-03	Resolución base	320 × 180
RT-04	Ventana	1280 × 720 (stretch <code>canvas_items</code> , aspect <code>expand</code>)
RT-05	Escena inicial	<code>res://scenes/menu.tscn</code>
RT-06	Autoloads globales	GameManager, DialogueManager, SceneTransition, etc.
RT-07	Control	Teclado (WASD, E, J, Q, ESC)

3. Descripción del juego

3.1 Género y público

- **Género:** aventura 2D top-down, exploración ligera, narrativa educativa.
- **Público:** adolescentes y público general en contextos escolares o de sensibilización.
- **Duración estimada:** 25–40 minutos la campaña completa.

3.2 Historia y simbolismo

Zona	Nombre	Simbolismo
1	Laberinto de Palabras	Rumores, insultos, laberinto mental
2	El Pantano	Lentitud, agotamiento, paciencia para avanzar
3	Cueva del Espejo	Autocrítica, culpa, reflejo del propio comportamiento
4	El Río	Fluir hacia el cambio, resistencia del entorno
Final	El Claro	Reencuentro honesto con Mateo

La Voz (narrador en cinemáticas) guía a Alex sin ser un personaje visible en mapa.

3.3 Sistemas principales

- **Ecos de luz:** coleccionables que abren salidas y dan puntos.
- **Empatía:** barra lógica (`empathy_level` en `GameManager`); sube al sanar al jefe y en momentos narrativos.
- **Sombras del acoso:** enemigos que representan hostigamiento; dañan pero pueden empujarse con el pulso.
- **Sin violencia obligatoria:** el final exige acercarse al jefe con manos vacías ([Q]) o interactuar ([E]).

3.4 Controles

Tecla	Acción	Uso pedagógico
WASD / Flechas	Mover	Exploración
E	Interactuar	Objetivos, sanar jefe
J	Pulso de luz	Defensa sin daño letal
Q	Gesto de paz	Renuncia explícita a la violencia
ESC	Pausa	Control del ritmo

4. Flujo de la campaña

MENÚ

[Jugar] → cinemática intro_campaign

▼

ZONA 1 – Laberinto (4 ecos → salida X)

cinemática bridge_1_2

▼

```
ZONA 2 – Pantano (3 pilares + 3 ecos → salida X)
|  cinemática bridge_2_3
▼
ZONA 3 – Cueva (cristales + sellos + ecos → jefe → sanar)
|  cinemática bridge_3_4
▼
ZONA 4 – Río (3 ecos → salida X)
|  cinemática bridge_4_clearing
▼
CLARO – Diálogo con Mateo → fin
```

Cada transición pasa por SceneTransition: cinemática (opcional) → fade negro → change_scene_to_file → fade in.

5. Mecánicas por nivel

Zona 1 — Laberinto de Palabras

- **Meta:** 4 ecos de luz.
- **Mecánica única:** oleadas de enemigos más rápidos tras recoger ecos.
- **Salida:** tile x abajo a la derecha cuando `echoes_collected == 4`.
- **Archivos:** `Zone1_Labyrinth.gd`, `Zone1_Labyrinth.tscn`.

Zona 2 — El Pantano

- **Meta:** 3 pilares + 3 ecos.
- **Mecánica única:** mantener **[E]** cerca del pilar (`PILLAR_HOLD_TIME`); barro reduce velocidad de Alex.
- **Enemigos:** color magenta/rosa para contrastar con el barro verde.
- **Archivos:** `Zone2_Swamp.gd`.

Zona 3 — Cueva del Espejo

- **Meta:** 3 cristales azules + 3 sellos dorados + 3 ecos + sanar jefe.
- **Mecánica única:** lectura de memorias, encendido de sellos, **Sala del Espejo** al completar requisitos.
- **Jefe:** `MirrorBossVisual.gd`; sanación con **[E]/[Q]** cerca del área del jefe.
- **Archivos:** `Zone3_Cave.gd`, `MirrorBossVisual.gd`.

Zona 4 — El Río

- **Meta:** 3 ecos.
- **Mecánica única:** agua que frena el movimiento.
- **Archivos:** `Zone4_Rio.gd`.

El Claro

- Diálogo ramificado o lineal con Mateo (según implementación en `clearing.gd`).
- Cierre narrativo de la campaña.

6. Arquitectura en Godot

6.1 Conceptos básicos de Godot (para explicar al jurado)

Concepto	Qué es	En Lost Eco
Nodo	Unidad básica del árbol de escena	Alex, paredes, HUD
Escena (.tscn)	Árbol de nodos guardado	menu.tscn, Zone1_Labyrinth.tscn
Script (.gd)	Comportamiento en GDScript	Alex.gd, Zone1_Labyrinth.gd
Autoload	Nodo global siempre cargado	GameManager, DialogueManager
Señal	Evento entre nodos	health_changed, action_performed
CanvasLayer	Capa de UI independiente	HUD, mensajes, pausa
CharacterBody2D	Cuerpo con física 2D	Alex
Area2D	Detección de solapamiento	Ecos, daño de enemigos

6.2 Estructura del repositorio

```
lost-eco/
├─ autoloads/      → Servicios globales (singletons)
├─ scenes/         → Menú, zonas, jugador
├─ scripts/
│   ├─ mapa/       → Lógica de niveles, cámara, niebla
│   ├─ personaje/  → Alex, sprites, visual gótico
│   ├─ enemigos/   → IA y apariencia de sombras
│   ├─ puntuacion/ → HUD, iconos de vida
│   └─ menu/       → Estados del menú
├─ assets/         → Sprites, shaders
└─ project.godot   → Configuración e input
```

6.3 Autoloads (corazón del sistema global)

Autoload	Responsabilidad
GameManager	Vidas, puntos, empatía, pausa, game over, referencia a player
DialogueManager	Intros, avisos, reflexiones en pantalla
SceneTransition	Fade y cambio de escena
ZoneCinematicDirector	Diapositivas entre niveles
StoryReflections	Textos narrativos por id
QuestManager	Progreso zone1...zone4
SettingsManager	Opciones y orden de campaña
MenuStateMachine	Estados del menú (patrón State)
AudioManager / UISounds	Audio

Registrados en `project.godot` → sección `[autoload]`.

6.4 Capas visuales (orden Z)

Capa	Aprox. layer	Contenido
Mundo	0	Mapa, Alex, enemigos
Niebla	88	Shader sigue al jugador
Viñeta	92	Oscurecimiento bordes
HUD	100	Vidas, objetivos
Minimapa	101	Mapa pequeño
Mensajes	110	Diálogos
Cinemática	128	Entre niveles
Pausa / Game Over	120–125	Bloqueo de juego

7. Cómo funciona el código

7.1 Ciclo de vida al iniciar el juego

1. Godot carga `menu.tscn` (configurado en `project.godot`).

2. El jugador pulsa **Jugar** → `MenuStateMachine.transition_to_game()`.
3. Se resetean vidas y puntos; `SceneTransition.change_scene_with_cinematic("intro_campaign", Zone1)`.
4. Tras la cinemática, fade y carga `Zone1_Labyrinth.tscn`.
5. `Zone1_Labyrinth._ready()` genera mapa, enemigos, HUD; `ZoneMissionBriefs.show_for_zone(1)` explica la misión.
6. `Alex._ready()` asigna `GameManager.player = self`.
7. `call_deferred("_finish_player_setup")` coloca a Alex en la celda s del MAP.

Por qué `call_deferred`: el padre (zona) ejecuta `_ready` antes que el hijo (Alex). Si colocamos al jugador en el mismo frame, a veces `GameManager.player` aún es `null`.

7.2 Cómo se construye un mapa (data-driven)

En cada `ZoneX*.gd` hay una constante `MAP`: array de strings. Cada carácter = un tile de 16×16 px.

```
# Ejemplo simplificado
for y in MAP.size():
    for x in MAP[y].length():
        match MAP[y][x]:
            "#": crear pared con StaticBody2D
            "E": spawn eco
            "S": punto de aparición de Alex
            ".": suelo
```

Ventaja para la sustentación: «*El diseño de nivel es editable como texto; no hace falta recompilar arte para mover un eco.*»

7.3 Alex — movimiento y estados

Archivo: `scripts/personaje/Alex.gd`

- `_physics_process`: lee `Input.get_vector` con acciones `move_left`, `move_right`, etc.
- `move_and_slide()` aplica velocidad.
- `can_move`: si es `false`, no se mueve (intro, pausa, cinemática, game over).
- `take_hit(from_pos)`: retroceso y feedback al recibir daño.
- `peace_gesture()`: emite señal `action_performed("drop_weapon")` para que `Zone3` reaccione.

7.4 Daño y game over

1. `Area2D` del enemigo detecta cuerpo del jugador (`body_entered`).
2. La zona llama `GameManager.take_damage()`.
3. `current_health` baja; se emite `health_changed`.
4. El HUD actualiza corazones.
5. Si `current_health == 0` → pantalla GAME OVER con reintentar o menú.

7.5 Diálogos e intro de misión

- `DialogueManager.show_zone_intro(título, objetivo, hint, color)` crea panel centrado; bloquea movimiento hasta [E]/ESC.
- `ZoneMissionBriefs.show_for_zone(n)` centraliza los textos pedagógicos por zona.
- `show_corner_notice` para avisos cortos sin bloquear tanto.

7.6 Transiciones y cinemáticas

`SceneTransition._run_scene_change`:

1. Si hay `cinematic_id` → `await ZoneCinematicDirector.play(id)`.
2. `fade_out` → pantalla negra.
3. `get_tree().change_scene_to_file(ruta)`.
4. `fade_in`.

`ZoneCinematicDirector` lee diapositivas del diccionario `CINEMATICS` (título, cita, capítulo) — contenido separado del código de lógica.

7.7 Bootstrap de zona (evitar código duplicado)

Clase	Función
<code>ZoneVisualBootstrap</code>	Coloca a Alex en S, activa <code>GothicAtmosphere</code> , visual gótico
<code>ZoneUIBootstrap</code>	Crea <code>PremiumZoneHUD</code> y <code>ZoneMinimap</code>
<code>ShadowEnemyVisual.create</code>	Crea nodo enemigo con paleta de colores
<code>EnemyBehavior</code>	Patrulla, persecución, embestidas

7.8 Zona 3 — flujo del jefe (punto fuerte de la sustentación)

1. Contadores: `memories_read`, `seals_lit`, `echoes_collected`.
2. `_try_unlock_boss()` cuando los tres llegan al máximo.
3. `_run_boss_arena_sequence()`: oculta enemigos, overlay oscuro, cámara al jefe, teleport cerca del altar.
4. Jugador usa [E] o [Q] → `_heal_boss()` → animación → `_zone_complete()` → cinemática a Zona 4.

Mensaje clave: «El jefe no se “mata”; se sana. Eso codifica la respuesta no violenta.»

8. Patrones de diseño

Patrón	Ubicación	Explicación breve para el jurado
Singleton (Autoload)	GameManager, DialogueManager, ...	Una sola instancia global accesible desde cualquier escena
State	MenuStateMachine + <code>menu/states/</code>	El menú cambia entre principal, opciones y pausa sin if infinitos
Observer	Señales <code>health_changed</code> , <code>action_performed</code>	La UI se actualiza sola cuando cambia la vida o el jugador hace un gesto
Factory / Bootstrap	ZoneUIBootstrap, ShadowEnemyVisual, PlayerAnimationFactory	Creación uniforme de objetos complejos
Data-driven	MAP, CINEMATICS, StoryReflections	Datos (mapa, textos) separados de la lógica
Strategy (ligero)	GothicAtmosphere temas, EnemyBehavior modos	Comportamiento intercambiable según zona

Nota honesta para la defensa: no usamos MVC estricto. Es arquitectura típica de Godot: **escena + script por nivel + servicios globales**. Es válido y mantenible para un proyecto académico de este tamaño.

9. Demostración sugerida

Paso	Qué mostrar	Qué decir
1	Menú → Jugar	Intro cinemática y fade
2	Zona 1 — intro misión	Pasos numerados, HUD
3	Recoger 1–2 ecos	Reflexión narrativa + pulso [J]
4	(Opcional) Recibir daño	Vidas en HUD, game over si quieren verlo rápido
5	Zona 2 — pilares	Mantener [E], contraste enemigos magenta
6	Zona 3 — cristal y sello	Progreso en HUD (CRIST / LUCES / ECOS)
7	Desbloquear jefe	Sala del Espejo, sanar con E o Q
8	Claro (si hay tiempo)	Cierre con Mateo

Tiempo total recomendado: 8–12 minutos con explicación + 5 minutos preguntas.

10. Preguntas frecuentes del jurado

¿Por qué Godot y no Unity/Unreal?

Godot es libre, liviano y excelente para 2D pixel-art. GDScript es accesible para equipos académicos.

¿Cómo garantizan el mensaje anti-bullying?

Textos en `StoryReflections`, mecánica de sanación del jefe, gesto de paz [Q], y reflexiones al recoger ecos.

¿Se puede extender el juego?

Sí: añadir filas al `MAP`, nuevas entradas en `StoryReflections`, otra escena en `CAMPAIGN_SCENES`.

¿Qué fue lo más difícil técnicamente?

Orden de inicialización (`call_deferred`), capas de UI, y sincronizar cinemáticas con cambio de escena sin bloquear al jugador.

¿Hay pruebas?

Pruebas manuales por zona; el proyecto es presentación educativa, no suite automatizada extensa.

¿Accesibilidad?

Opciones de daltonismo en settings; textos grandes en intro; resolución escalable.

Anexo — Archivos clave para mostrar en pantalla

Tema	Archivo
Configuración global	<code>project.godot</code>
Vidas y game over	<code>autoloads/GameManager.gd</code>
Misión por zona	<code>scripts/mapa/ZoneMissionBriefs.gd</code>
Mapa zona 1	<code>scripts/mapa/Zone1_Labyrinth.gd</code> → <code>MAP</code>
Jugador	<code>scripts/personaje/Alex.gd</code>
Patrón State menú	<code>autoloads/MenuStateMachine.gd</code>
Cinemáticas	<code>autoloads/ZoneCinematicDirector.gd</code>
Documentación técnica extendida	<code>DOCUMENTACION_CODIGO.md</code> / <code>.pdf</code>